

BUSN 41210: Financial Analytics

Lecture 3: Diagnostics and Transformations, Categorical Variables, Interactions, Multiple Linear Regressions

Dacheng Xiu

Chicago Booth

September 11, 2026

SLR Model Assumptions

$$Y_i|X_i \stackrel{\text{ind}}{\sim} N(\beta_0 + \beta_1 X_i, \sigma^2)$$

Recall the key assumptions of our linear regression model:

- (i) The conditional mean of Y is **linear** in X .
- (ii) The additive errors (deviations from line)
 - ▶ are normally distributed
 - ▶ **independent** from each other
 - ▶ identically distributed (i.e., they have **constant variance**)

SLR Model Assumptions

Inference and prediction relies on this model being true!

If the model assumptions do not hold, then all bets are off:

- ▶ prediction can be systematically biased
- ▶ standard errors, intervals, and t -tests are wrong

We will focus on using graphical methods (plots!) to detect violations of the model assumptions.

Example Model Violations

Anscombe's quartet comprises four datasets that have similar statistical properties...

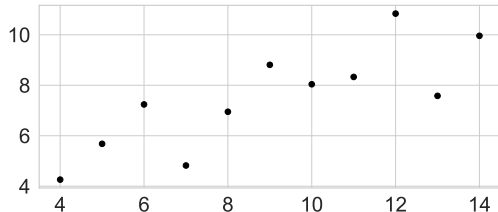
```
> for c in ['1', '2', '3', '4']:
    x, y = anscombe['x'+c], anscombe['y'+c]
    print(x.mean(), y.mean(), x.std(), y.std(),
          np.corrcoef(x, y)[0, 1])
```

Output:	mean(x)	mean(y)	sd(x)	sd(y)	corr
set 1	9.0	7.5	3.317	2.032	0.816
set 2	9.0	7.5	3.317	2.032	0.816
set 3	9.0	7.5	3.317	2.030	0.816
set 4	9.0	7.5	3.317	2.031	0.817

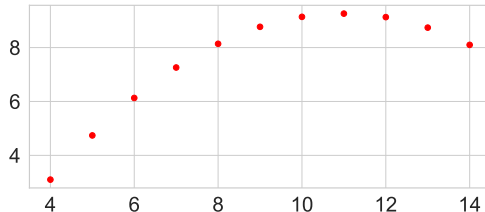
Example Model Violations

...but vary considerably when graphed:

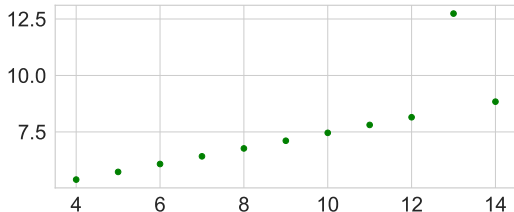
Dataset 1



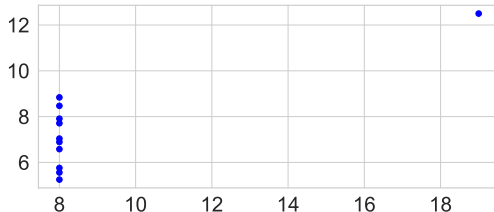
Dataset 2



Dataset 3



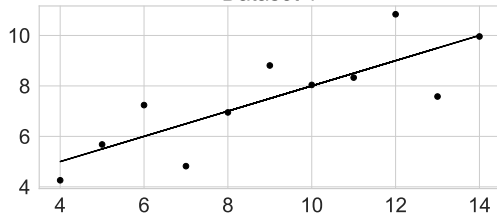
Dataset 4



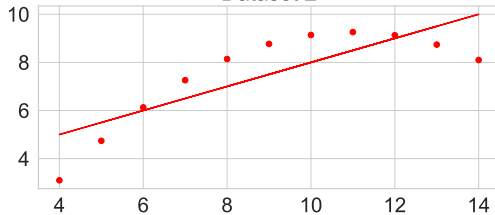
Example Model Violations

Similarly, let's consider linear regression for each dataset.

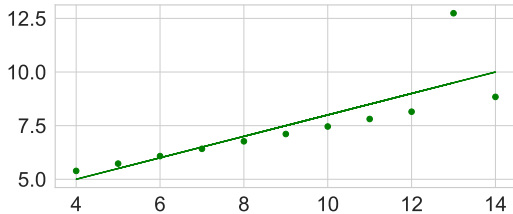
Dataset 1



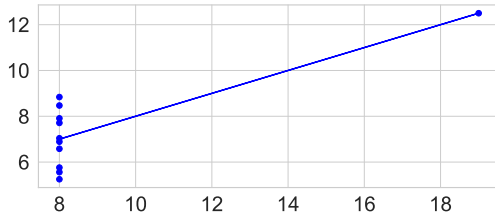
Dataset 2



Dataset 3



Dataset 4



Example Model Violations

The regression lines and R^2 values are the same...

```
# Store models in a list
> ansreg = [smf.ols(formula=f'y{i} ~ x{i}', data=anscombe).fit() for i in range(1, 5)]

# Get the coefficients for each regression model
> coefs = np.column_stack([reg.params for reg in ansreg])
> coefs_rounded = np.round(coefs, 1)
> coefs_rounded
```

Output:

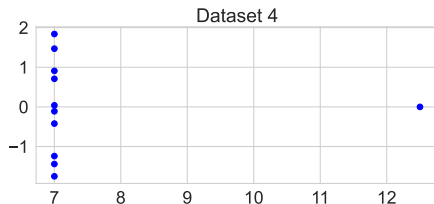
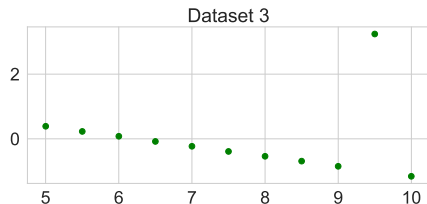
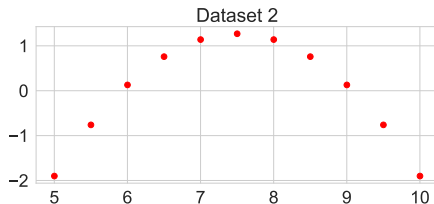
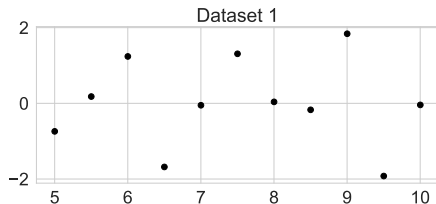
```
[[3.0 3.0 3.0 3.0]
 [0.5 0.5 0.5 0.5]]
```

```
# Apply summary to each model and get R-squared values
> smry = [reg.rsquared for reg in ansreg]
> r_squared_rounded = np.round(smry, 1)
> r_squared_rounded
```

Output: [0.7, 0.7, 0.7, 0.7]

Example Model Violations

...but the residuals (plotted against $\hat{Y}$) look totally different.



Plotting e vs $\hat{Y}$ is your #1 tool for finding fit problems.

Residuals and the Model Assumptions

Recall that the linear regression model assumes

$$Y_i = \beta_0 + \beta_1 X_i + \varepsilon_i, \text{ where } \varepsilon_i \stackrel{iid}{\sim} N(0, \sigma^2)$$

Our goal is to determine if the “true” residuals are *iid* normal and unrelated to X . If the SLR model assumptions are true, then the residuals must be just “white noise”:

- (i) Each ε_i has the same variance (σ^2).
- (ii) Each ε_i has the same mean (0).
- (iii) All of the ε_i have the same normal distribution.

Residuals and the Model Assumptions

Since the true ε_i residuals are unknown, we must look instead at the least squares estimated residuals.

We estimate $Y_i = b_0 + b_1 X_i + e_i$, such that the sample least squares regression residuals are $e_i = Y_i - \hat{Y}_i$.

What should the e_i residuals look like if the SLR model is true?

Residuals and the Model Assumptions

If the SLR model is true, it turns out that:

$$e_i \sim N(0, \sigma^2[1 - h_i]), \quad h_i = \frac{1}{n} + \frac{(X_i - \bar{X})^2}{\sum_{j=1}^n (X_j - \bar{X})^2}.$$

The h_i term is referred to as the i^{th} observation's **leverage**:

- It is that point's share of the data ($1/n$) plus its proportional contribution to variability in X .
- As $n \rightarrow \infty$, $h_i \rightarrow 0$ and residuals become $\varepsilon_i \sim N(0, \sigma^2)$.

Understanding Leverage

The h_i leverage term measures sensitivity of the estimated least squares regression line to changes in Y_i .

The term “leverage” provides a mechanical intuition:

- The farther you are from a pivot joint, the more torque you have pulling on a lever.

Outliers do more damage if they have high leverage!

Standardized Residuals

Since $e_i \sim N(0, \sigma^2[1 - h_i])$, we know that

$$\frac{e_i}{\sigma\sqrt{1 - h_i}} \sim N(0, 1).$$

These transformed e_i 's are called the **Standardized Residuals**.

They all have the same distribution **if** the SLR model is true.

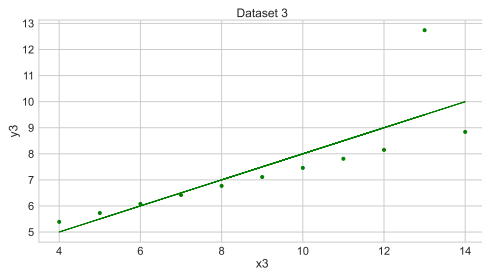
They are almost (close enough) independent ($\overset{iid}{\sim} N(0, 1)$).

Standardized Residuals

As always, we don't know $\text{sd}(\varepsilon) = \sigma$.

Usually, we estimate $\sigma^2 \approx s^2 = \frac{1}{n-p} \sum_{j=1}^n e_j^2$ ($p = 2$ for SLR).

Here, we want to see whether any particular e_i is “too big”, and you don't want a single outlier to make s artificially large.



One big outlier can make s overestimate σ

Studentized Residuals

We thus define a Studentized Residual as

$$r_i = \frac{e_i}{s_{-i}\sqrt{1-h_i}} \sim t_{n-p-1}(0, 1)$$

where $s_{-i}^2 = \frac{1}{n-p-1} \sum_{j \neq i} e_j^2$ is $\hat{\sigma}$ calculated without e_i .

These are easy to get in Python with the `OLSInfluence().resid_studentized_external` function.

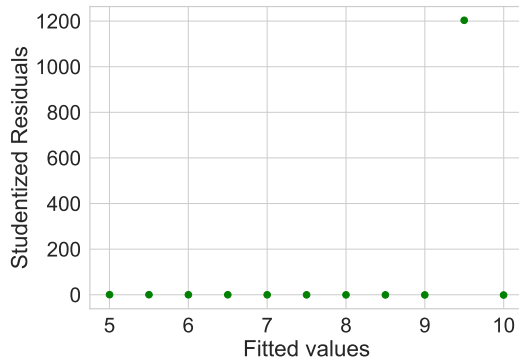
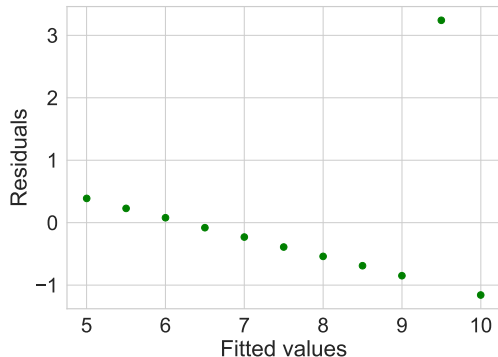
```
> OLSInfluence(reg1).resid_studentized_external
```

```
Output:  0          1          2          3          4
         0.031345 -0.040845 -2.081099  1.126800 -0.139801
         5          6          7          8          9
        -0.038196  1.116959 -0.704581  1.838330 -1.568460
        10
        0.156809
```

Outliers and Studentized Residuals

Since the studentized residuals are distributed $t_{n-p-1}(0, 1)$, we should be concerned about any r_i outside of about $[-2.5, 2.5]$.

Consider the 3rd Anscombe dataset:



How to Deal with Outliers

When should you delete outliers?

Only when you have a really good reason!

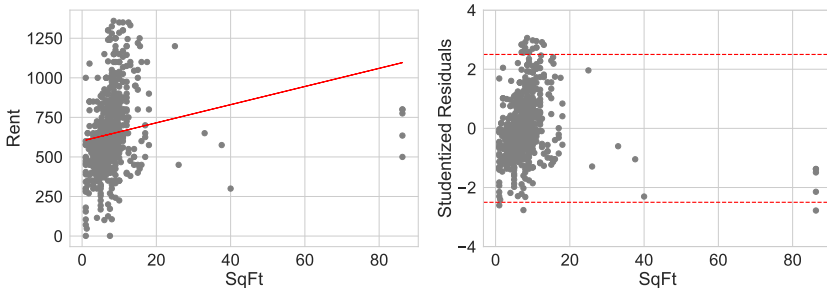
There is nothing wrong with running regression with and without potential outliers to see whether results are significantly impacted.

Any time outliers are dropped, the reasons for doing so should be clearly noted.

Outliers, Leverage, and Residuals

Warning: Unfortunately, outliers with high leverage are hard to catch through r_i (since the line is pulled towards them).

Consider data on house Rents vs SqFt:



Plots of r_i or e_i against $\hat{Y}_i$ or X_i are still your best diagnostic!

Beyond SLR

Many problems involve more than one independent variable or factor which affects the dependent or response variable.

- ▶ Multi-factor asset pricing models (beyond CAPM)
- ▶ Demand for a product given prices of competing brands, advertising, household attributes, etc.
- ▶ More than size to predict house price!

In SLR, the conditional mean of Y depends on X . The Multiple Linear Regression (MLR) model extends this idea to include more than one independent variable.

The MLR Model

In general, the MLR model is same as always, but with more covariates.

$$Y|X_1 \dots X_d \overset{ind}{\sim} N(\beta_0 + \beta_1 X_1 \dots + \beta_d X_d, \sigma^2)$$

Recall the key assumptions of our linear regression model:

- (i) The conditional mean of Y is **linear** in the X_j variables.
- (ii) The additive errors (deviations from line)
 - ▶ are normally distributed
 - ▶ **independent** from each other
 - ▶ identically distributed (i.e., they have **constant variance**)

The MLR Model

Our interpretation of regression coefficients can be extended from the simple single covariate regression case:

$$\beta_j = \frac{\partial \mathbb{E}[Y|X_1, \dots, X_d]}{\partial X_j}$$

Holding all other variables constant,
 β_j is the average change in Y per unit change in X_j .

The Data and Least Squares

The data in multiple regression is a set of points with values for output Y and for each input variable.

Data: Y_i and $\mathbf{x}_i = [X_{1i}, X_{2i}, \dots, X_{di}]$, for $i = 1, \dots, n$.

Or, as a data array (i.e., **DataFrame**),

$$\text{Data} = \begin{bmatrix} Y_1 & X_{11} & X_{21} & \dots & X_{d1} \\ Y_2 & X_{12} & X_{22} & \dots & X_{d2} \\ & & \vdots & & \\ Y_n & X_{1n} & X_{2n} & \dots & X_{dn} \end{bmatrix}$$

The Data and Least Squares

$$Y = \beta_0 + \beta_1 X_1 \dots + \beta_d X_d + \varepsilon, \quad \varepsilon \sim N(0, \sigma^2)$$

How do we estimate the MLR model parameters?

The principle of Least Squares is exactly the same as before:

- ▶ Define the fitted values
- ▶ Find the best fitting plane by minimizing the sum of squared residuals.

The Data and Least Squares

Fitted Values: $\hat{Y}_i = b_0 + b_1X_{1i} + b_2X_{2i} \dots + b_dX_{di}$.

Residuals: $e_i = Y_i - \hat{Y}_i$.

Standard Error: $s = \sqrt{\frac{\sum_{i=1}^n e_i^2}{n - p}}$, where $p = d + 1$.

Least Squares: Find $b_0, b_1, b_2, \dots, b_d$ to minimize s^2 .

The Data and Least Squares

What are the LS coefficient values? Say that

$$\mathbf{Y} = \begin{bmatrix} Y_1 \\ \vdots \\ Y_n \end{bmatrix} \quad \hat{\mathbf{X}} = \begin{bmatrix} 1 & X_{11} & X_{12} & \dots & X_{1d} \\ & & \vdots & & \\ 1 & X_{n1} & X_{n2} & \dots & X_{nd} \end{bmatrix}$$

Then the estimates are $[b_0 \dots b_d]' = \mathbf{b} = (\hat{\mathbf{X}}'\hat{\mathbf{X}})^{-1}\hat{\mathbf{X}}'\mathbf{Y}$.

Same intuition as for SLR: $\mathbf{b}$ captures the covariance between X_j and Y ($\hat{\mathbf{X}}'\mathbf{Y}$), normalized by input sum of squares ($\hat{\mathbf{X}}'\hat{\mathbf{X}}$).

Regression in Python

You need only one command

```
reg = smf.ols(y ~ var1 + ... + varP, data=mydata).fit()
```

ols stands for Ordinary Least Squares.

$y \sim a + b$ is the 'formula' that defines your regression.

The object `reg` is a list of useful things.

`reg.summary()` prints a bunch of information.

`reg.params` gives coefficients.

`reg.predict(mynewdata)` predicts.

`mynewdata` must be a data frame with exactly the same format as `mydata` (same variable names, same factor levels).

Example: Predicting Market Equity Premium

We will focus on predicting market equity returns using a dataset of financial and macroeconomic indicators compiled by Welch and Goyal (2007).

Data: 16 predictors and includes 74 annual observations, covering a period from 1948 to 2021.

The following link gives more details on this data set.

[Market Equity Premium Data Set](#)

F1.head(10)

	dfy	infl	svar	d_e	lty	tms	tbi	dfr	d_p	d_y	ltr	e_p	b_m	ik	ntis	eqis	ret
0	0.0066	0.088372	0.019339	-0.650588	0.0243	0.0148	0.0095	0.002911	-2.902206	-2.902206	-0.026275	-2.251619	0.725326	0.034505	0.025922	0.239630	0.033241
1	0.0074	0.029915	0.022100	-0.901123	0.0237	0.0121	0.0116	0.007509	-2.793866	-2.800424	0.033850	-1.892744	0.840948	0.034801	0.015015	0.157140	0.140497
2	0.0073	-0.020747	0.012306	-0.710539	0.0209	0.0099	0.0110	-0.031303	-2.687967	-2.590267	0.064391	-1.977428	0.796429	0.029279	0.027176	0.191869	0.243920
3	0.0053	0.059322	0.021461	-0.658542	0.0224	0.0090	0.0134	0.020647	-2.630763	-2.433733	0.000505	-1.972221	0.722538	0.034373	0.031358	0.227023	0.182102
4	0.0060	0.060000	0.012182	-0.548408	0.0269	0.0096	0.0173	0.012519	-2.824835	-2.672435	-0.039422	-2.276426	0.721316	0.032582	0.036151	0.264824	0.128976
5	0.0054	0.007547	0.006590	-0.531879	0.0279	0.0070	0.0209	0.023516	-2.936193	-2.824835	0.011693	-2.404314	0.694073	0.032615	0.026535	0.202497	-0.052586
6	0.0061	0.007491	0.008279	-0.548719	0.0274	0.0114	0.0160	-0.002144	-2.839683	-2.908219	0.036258	-2.290964	0.759701	0.033956	0.024822	0.203978	0.386943
7	0.0055	-0.007435	0.008390	-0.587065	0.0272	0.0157	0.0115	-0.017918	-3.151181	-2.779464	0.071776	-2.564116	0.604120	0.032514	0.026993	0.213197	0.243488
8	0.0047	0.003745	0.021620	-0.791778	0.0295	0.0041	0.0254	0.017632	-3.322576	-3.088267	-0.012834	-2.530799	0.509828	0.036554	0.025245	0.275391	0.011814
9	0.0062	0.029851	0.013890	-0.672827	0.0345	0.0024	0.0321	-0.012204	-3.289216	-3.263388	-0.055942	-2.616389	0.544177	0.035529	0.026149	0.268489	-0.175425

Summary: Market Equity Premium

```
> rest_full = F1.columns[0:-1].tolist()
> rest_full = '+'.join(rest_full)
> reg_full = smf.ols(formula='ret ~ {}'.format(rest_full), data = F1).fit()
> reg_full.summary()
```

OLS Regression Results						
Dep. Variable:	ret			R-squared:	0.419	
Model:	OLS			Adj. R-squared:	0.281	
Method:	Least Squares			F-statistic:	3.040	
Date:	Fri, 13 Sep 2024			Prob (F-statistic):	0.00142	
Time:	13:35:35			Log-Likelihood:	45.512	
No. Observations:	74			AIC:	-61.02	
Df Residuals:	59			BIC:	-26.46	
Df Model:	14					
Covariance Type:	nonrobust					
	coef	std err	t	P> t	[0.025	0.975]
Intercept	1.1315	0.456	2.481	0.016	0.219	2.044
dfy	9.4915	7.093	1.338	0.186	-4.701	23.684
infl	-2.0196	1.094	-1.846	0.070	-4.209	0.170
svar	1.5749	0.960	1.640	0.106	-0.347	3.496

d_e	-0.0248	0.069	-0.360	0.720	-0.163	0.113
lty	-0.8330	0.719	-1.158	0.251	-2.272	0.606
tms	-0.1880	1.067	-0.176	0.861	-2.324	1.948
tbl	-0.6451	0.792	-0.815	0.418	-2.229	0.939
dfr	0.3791	0.562	0.675	0.502	-0.745	1.503
d_p	0.1297	0.119	1.088	0.281	-0.109	0.368
d_y	-0.0127	0.150	-0.084	0.933	-0.314	0.288
ltr	0.1221	0.227	0.538	0.592	-0.332	0.576
e_p	0.1545	0.086	1.800	0.077	-0.017	0.326
b_m	-0.0574	0.229	-0.250	0.803	-0.516	0.401
ik	-2.8554	9.049	-0.316	0.753	-20.963	15.252
ntis	2.1022	1.319	1.594	0.116	-0.536	4.741
eqis	-0.9558	0.337	-2.833	0.006	-1.631	-0.281

Multiple vs Simple Linear Regression

Basic concepts and techniques translate directly from SLR.

- ▶ Individual parameter inference and estimation is the same, conditional on the rest of the model.
- ▶ ANOVA is exactly the same, and the F -test still applies.
- ▶ Our diagnostics and transformations apply directly.

The hardest part would be moving to matrix algebra to translate all of our equations.

Luckily, Python does all that for you.

Residual Standard Error

First off, the calculation for $s^2 = \text{var}(e)$ is exactly the same:

$$s^2 = \frac{\sum_{i=1}^n (Y_i - \hat{Y})^2}{n - p}$$

- ▶ $\hat{Y}_i = b_0 + \sum b_j X_{ji}$ and $p = d + 1$.
- ▶ The residual standard error is $\hat{\sigma} = s = \sqrt{s^2}$.

Residuals in MLR

As in the SLR model, the residuals in multiple regression are purged of any relationship to the independent variables. We decompose Y into the part predicted by X and the part due to idiosyncratic error.

$$Y = \hat{Y} + e$$

$$\text{corr}(X_j, e) = 0 \qquad \text{corr}(\hat{Y}, e) = 0$$

Inference for Coefficients

As before in SLR, the LS linear coefficients are random (different for each sample) and correlated with each other.

The LS estimators are unbiased: $\mathbb{E}[b_j] = \beta_j$ for $j = 0, \dots, d$.

In particular, the sampling distribution for $\mathbf{b}$ is a multivariate normal, with mean $\boldsymbol{\beta} = [\beta_0 \cdots \beta_d]'$ and covariance matrix $\mathbf{S}_b$.

$$\mathbf{b} \sim N(\boldsymbol{\beta}, \mathbf{S}_b)$$

Coefficient Covariance Matrix

$\text{var}(\mathbf{b})$: the $p \times p$ covariance matrix for random vector $\mathbf{b}$ is

$$\mathbf{S}_b = \begin{bmatrix} \text{var}(b_0) & \text{cov}(b_0, b_1) & & & \\ \text{cov}(b_1, b_0) & \text{var}(b_1) & & & \\ & & \ddots & & \\ & & & \text{var}(b_{d-1}) & \text{cov}(b_{d-1}, b_d) \\ & & & \text{cov}(b_d, b_{d-1}) & \text{var}(b_d) \end{bmatrix}$$

$\Rightarrow$ Standard errors are the square root of the diagonal of $\mathbf{S}_b$.

The coefficient covariance matrix is “easy” to calculate:

$$\mathbf{S}_b = s^2 \left(\hat{\mathbf{X}}' \hat{\mathbf{X}} \right)^{-1}$$

Coefficient Covariance Matrix

You can get Covariance Matrix with `cov_params()`

```
> cov_matrix = reg_full.cov_params()
> cov_matrix
```

	Intercept	dfy	infl	svar	d_e	lty	tms	tbl	dfr	d_p	d_y	ltr	e_p	b_m	ik	ntis	eqis
Intercept	0.208015	0.288130	0.067342	-0.019202	0.008613	-0.003088	-0.100265	0.097176	0.013308	0.031949	-0.016038	0.008569	0.023335	-0.079991	-1.313825	0.043282	0.002252
dfy	0.288130	50.304174	-1.291776	-2.468460	-0.098280	-2.235363	-1.846633	-0.388730	0.627243	0.035913	0.134162	-0.213049	0.134194	-0.554110	19.496972	4.607925	-0.498247
infl	0.067342	-1.291776	1.196859	-0.196164	0.015424	-0.087064	0.311195	-0.398259	-0.052503	0.000393	0.021307	0.040273	-0.015031	-0.070906	-0.255213	-0.244820	0.085364
svar	-0.019202	-2.468460	-0.196164	0.922117	-0.018056	0.070516	-0.061330	0.131846	0.048032	-0.016341	0.022270	0.015681	0.001715	0.031731	0.497380	0.058751	-0.029064
d_e	0.008613	-0.098280	0.015424	-0.018056	0.004740	0.002316	-0.001165	0.003481	0.011523	0.005789	-0.005768	0.003183	0.001049	-0.003492	0.020232	-0.014237	-0.000492
lty	-0.003088	-2.235363	-0.087064	0.070516	0.002316	0.517376	0.514876	0.002500	-0.152023	-0.004169	-0.001227	-0.046972	-0.006485	0.022788	-1.403594	-0.161890	-0.005426
tms	-0.100265	-1.846633	0.311195	-0.061330	-0.001165	0.514876	1.139112	-0.624236	-0.220128	-0.019237	0.033787	-0.057226	-0.018072	0.023234	2.123407	-0.270313	0.035737
tbl	0.097176	-0.388730	-0.398259	0.131846	0.003481	0.002500	-0.624236	0.626736	0.068105	0.015068	-0.035014	0.010254	0.011587	-0.000446	-3.527001	0.108423	-0.041163
dfr	0.013308	0.627243	-0.052503	0.048032	0.011523	-0.152023	-0.220128	0.068105	0.315471	0.028588	-0.030200	0.072666	0.017066	-0.019740	1.378884	0.089414	-0.019374
d_p	0.031949	0.035913	0.000393	-0.016341	0.005789	-0.004169	-0.019237	0.015068	0.028588	0.014205	-0.014592	0.008415	0.008416	-0.015924	0.066611	0.012717	0.001911
d_y	-0.016038	0.134162	0.021307	0.022270	-0.005768	-0.001227	0.033787	-0.035014	-0.030200	-0.014592	0.022600	-0.009207	-0.008824	0.005586	0.308598	-0.001977	-0.006118
ltr	0.008569	-0.213049	0.040273	0.015681	0.003183	-0.046972	-0.057226	0.010254	0.072666	0.008415	-0.009207	0.051428	0.005232	-0.007720	0.250425	0.027396	0.003979
e_p	0.023335	0.134194	-0.015031	0.001715	0.001049	-0.006485	-0.018072	0.011587	0.017066	0.008416	-0.008824	0.005232	0.007367	-0.012432	0.046379	0.026954	0.002404
b_m	-0.079991	-0.554110	-0.070906	0.031731	-0.003492	0.022788	0.023234	-0.000446	-0.019740	-0.015924	0.005586	-0.007720	-0.012432	0.052580	-0.300959	-0.062356	-0.017270
ik	-1.313825	19.496972	-0.255213	0.497380	0.020232	-1.403594	2.123407	-3.527001	1.378884	0.066611	0.308598	0.250425	0.046379	-0.300959	81.888835	1.986561	-0.427842
ntis	0.043282	4.607925	-0.244820	0.058751	-0.014237	-0.161890	-0.270313	0.108423	0.089414	0.012717	-0.001977	0.027396	0.026954	-0.062356	1.986561	1.738448	-0.191072
eqis	0.002252	-0.498247	0.085364	-0.029064	-0.000492	-0.005426	0.035737	-0.041163	-0.019374	0.001911	-0.006118	0.003979	0.002404	-0.017270	-0.427842	-0.191072	0.113795

Inference for Individual Coefficients

Intervals and t -statistics are **exactly the same** as in SLR.

- ▶ A $(1 - \alpha)100\%$ C.I. for β_j is $b_j \pm t_{\alpha/2, n-p} s_{b_j}$.
- ▶ $z_{b_j} = (b_j - \beta_j^0) / s_{b_j} \sim t_{n-p}(0, 1)$ is the number of standard errors between the LS estimate and the null value.

Intervals and testing via b_j & s_{b_j} are **one-at-a-time procedures**:

- ▶ You are evaluating the j^{th} coefficient conditional on the other X 's being in the model, but **regardless of the values you've estimated for the other b 's**.

R^2 for multiple regression

Recall that we already view R^2 as a measure of “fit”:

$$R^2 = \frac{SSR}{SST} = \frac{\sum_{i=1}^n (\hat{Y}_i - \bar{Y})^2}{\sum_{i=1}^n (Y_i - \bar{Y})^2}$$

And the correlation interpretation remains:

$$R^2 = \text{cor}^2(\hat{Y}, Y) = r_{\hat{Y}Y}^2$$

($r_{\hat{Y}Y} = r_{XY}$ in SLR since $\text{cor}(X, \hat{Y}) = 1$.)

Forecasting in MLR

Prediction follows exactly the same methodology as in SLR.

For new data $\mathbf{x}_f = [X_{1,f} \cdots X_{d,f}]'$,

- ▶ $\mathbb{E}[Y_f|\mathbf{x}_f] = \hat{Y}_f = b_0 + b_1X_{1f} + \dots b_dX_{df}$
- ▶ $\text{var}[Y_f|\mathbf{x}_f] = \text{var}(\hat{Y}_f) + \text{var}(e_f) = s_{\text{fit}}^2 + s^2 = s_{\text{pred}}^2.$

With $\hat{\mathbf{X}}$ our design matrix and $\hat{\mathbf{x}}_f = [1, X_{1,f} \cdots X_{d,f}]'$

$$s_{\text{fit}}^2 = s^2 * \hat{\mathbf{x}}_f' (\hat{\mathbf{X}}' \hat{\mathbf{X}})^{-1} \hat{\mathbf{x}}_f$$

A $(1 - \alpha)$ level prediction interval is still $\hat{Y}_f \pm t_{\alpha/2, n-p} s_{\text{pred}}.$

Prediction in MLR

```
> reg_reduced = smf.ols(formula='ret ~ infl + lty + tbl', data=F1).fit()
> Xnew = pd.DataFrame([[0.03,0.02, 0.01]], columns=['infl', 'lty', 'tbl'])
> reg_reduced.get_prediction(Xnew).summary_frame(alpha=0.05)
```

mean	mean_se	mean_ci_lower	mean_ci_upper	obs_ci_lower	obs_ci_upper
0.091831	0.033244	0.025529	0.158134	-0.236458	0.42012

```
> X = sm.add_constant(X_reduced) # add 1
> Xf = np.array([1,0.03,0.02, 0.01]).T
> y_hat_f = Xf @ reg_reduced.params
y_hat_f: 0.0918

> n = len(X)
> p = len(reg_reduced.params)
> s2 = reg_reduced.scale
> s2_fit = s2 * (Xf.T @ np.linalg.inv(X.T @ X) @ Xf)
> s_pred = np.sqrt(s2 + s2_fit)
> t_critical = t.ppf(1 - 0.05 / 2, df=reg_reduced.df_resid)
> lower_bound = y_hat_f - t_critical * s_pred
> upper_bound = y_hat_f + t_critical * s_pred
Prediction Interval: [-0.2365, 0.4201]
```

Multicollinearity

Multicollinearity refers to strong linear dependence between some of the covariates in a multiple regression model.

The usual marginal effect interpretation is lost:

- Change in one X variable leads to change in others.

Coefficient standard errors will be large, such that multicollinearity leads to large uncertainty about b_j 's.

Regression coefficient estimates are unstable. Dropping or adding variables may cause significant changes in the rest coefficient estimates.

Multicollinearity

Suppose that you regress Y onto X_1 and $X_2 = 10 * X_1$.

Then $\mathbb{E}[Y] = \beta_0 + \beta_1 X_1 + \beta_2 X_2 = \beta_0 + \beta_1 X_1 + \beta_2 (10X_1)$ and the marginal effect of X_1 on Y is

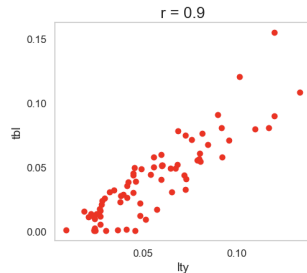
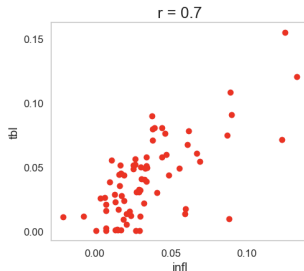
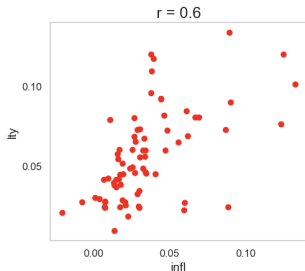
$$\frac{\partial \mathbb{E}[Y|X_1, X_2]}{\partial X_1} = \beta_1 + 10\beta_2$$

X_1 and X_2 do not act independently!

Multicollinearity

Let's consider 3 of our covariates:

- ▶ `infl`: Inflation
- ▶ `lty`: Long Term Yield
- ▶ `tb1`: Tresuary Bill



Sure enough, they are all correlated with each other.

Multicollinearity

In the 3 covariate regression, none of the effects are significant:

```
> reg_reduced = smf.ols(formula='ret ~ infl + lty + tbl', data=F1).fit()
> reg_reduced.summary()
```

```

=====
                        OLS Regression Results
=====
Dep. Variable:          ret      R-squared:                0.165
Model:                  OLS      Adj. R-squared:           0.130
Method:                 Least Squares      F-statistic:        4.627
Date:                   Thu, 29 Aug 2024      Prob (F-statistic):    0.00520
Time:                   17:21:51      Log-Likelihood:       32.108
No. Observations:       74      AIC:                  -56.22
Df Residuals:           70      BIC:                  -47.00
Df Model:                3
Covariance Type:        nonrobust
=====

```

	coef	std err	t	P> t	[0.025	0.975]
Intercept	0.1083	0.047	2.325	0.023	0.015	0.201
infl	-0.0007	0.910	-0.001	0.999	-1.816	1.815
lty	0.5056	1.428	0.354	0.724	-2.343	3.354
tbl	-2.6552	1.459	-1.820	0.073	-5.565	0.255

But the F -stat is significant with p -value = 0.005!

Multicollinearity

If you look at individual regression effects, all 3 are significant:

```
reg_infl = smf.ols(formula='ret ~ infl', data=F1).fit()  
print(reg_infl.summary() )
```

	coef	std err	t	P> t	[0.025	0.975]
Intercept	0.0879	0.030	2.902	0.005	0.028	0.148
infl	-1.6956	0.671	-2.526	0.014	-3.034	-0.358

```
reg_lty = smf.ols(formula='ret ~ lty', data=F1).fit()  
print(reg_lty.summary() )
```

	coef	std err	t	P> t	[0.025	0.975]
Intercept	0.1449	0.042	3.411	0.001	0.060	0.230
lty	-2.0887	0.685	-3.049	0.003	-3.454	-0.723

```
reg_tbl = smf.ols(formula='ret ~ tbl', data=F1).fit()  
print(reg_tbl.summary() )
```

	coef	std err	t	P> t	[0.025	0.975]
Intercept	0.1201	0.030	3.942	0.000	0.059	0.181
tbl	-2.2545	0.600	-3.758	0.000	-3.450	-1.059

Multicollinearity

Multicollinearity is not a big problem in and of itself, you just need to know that it is there.

- ▶ Fitting could remain good.
- ▶ No effect on prediction.

If you recognize multicollinearity:

- ▶ Understand that the β_j are not true marginal effects.
- ▶ Consider dropping variables to get a more simple model (use the partial F -test!).
- ▶ Expect to see big standard errors on your coefficients (i.e., your coefficient estimates are unstable).

Categorical Effects and Dummy Variables

Regression for Y onto a categorical X that takes R different levels:

$$\mathbb{E}[Y | X = 0] = \beta_0; \quad \mathbb{E}[Y | X = r] = \beta_0 + \beta_r, \text{ for } r = 1, \dots, R - 1$$

To represent these qualitative factors in multiple regression, we use **dummy**, **binary**, or **indicator** variables.

Dummy variables allow the mean (intercept) to shift by taking on the value 0 or 1.

If a factor X takes R possible levels, we can represent X through $R - 1$ dummy variables

$$\mathbb{E}[Y|X] = \beta_0 + \beta_1 \mathbb{1}_{[X=1]} + \beta_2 \mathbb{1}_{[X=2]} \dots + \beta_{R-1} \mathbb{1}_{[X=R-1]}$$

($\mathbb{1}_{[X=r]} = 1$ if $X = r$, 0 if $X \neq r$.) **What is $\mathbb{E}[Y|X]$ if $X = 1$?**



Orange Juice

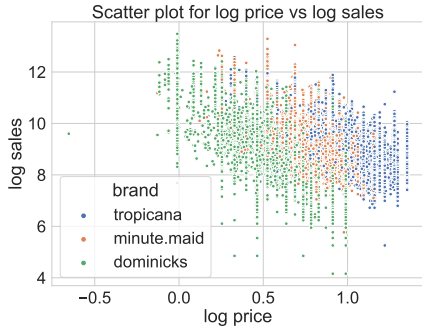
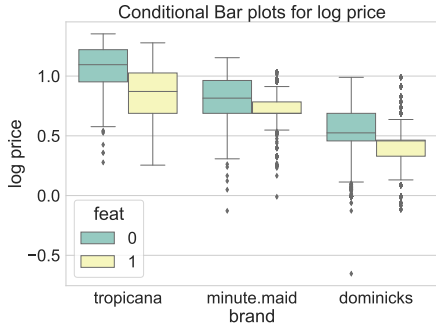
Three brands (*b*) Tropicana,
Minute Maid, Dominicks

83 Chicagoland Stores
Demographic info for each

Price, sales (log of # units sold),
and whether advertised (feat)

data in oj.csv

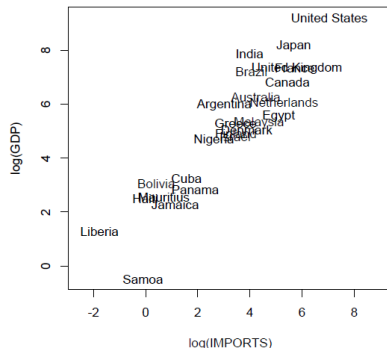
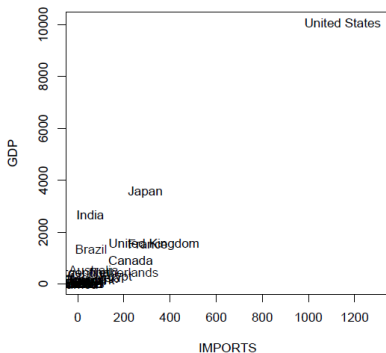
The Juice: price, brand, and sales



Each brand occupies a well defined price range. Sales decrease with price.

Thinking About Scale

When making a linear point (this goes up, that goes down) think about the scale on which you expect to find linearity.



If your scatterplots look like the left panel, consider using **log**. Outliers?

Log Linear

We often model the mean for $\log(y)$ instead of y .

Why? Multiplicative (rather than additive) change.

$$\log(y) = \log(a) + x\beta \Leftrightarrow y = ae^{x\beta}.$$

Predicted y is multiplied by e^β after a unit increase in x .

Recall that $\log(y) = z \Leftrightarrow e^z = y$ where $e \approx 2.7$

$\log(ab) = \log(a) + \log(b)$ and $\log(a^b) = b \log(a)$.

I use $\log = \ln$, natural log. Anything else will be noted, e.g., $\log_2$.

Whenever y changes on a percentage scale, use $\log(y)$.

prices: "... Foreclosed homes sell at a 20% to 30% discount"

sales: "... our y.o.y. sales are up 20% across models"

Price Elasticity

A simple orange juice 'elasticity model' for sales y has

$$\mathbb{E}[\log y] = \gamma \log(\text{price}) + \mathbf{x}'\beta$$

Elasticities and log-log regression: for small values we can interpret γ as % change in y per 1% increase in price.

We run this in Python:

```
smf.ols(formula='log_sales ~ log_price+brand',data=oj).fit()  
(Intercept) log(price) branBDinute.maid brandtropicana  
10.8288      -3.1387           0.8702           1.5299
```

and see sales drop by about 3.1% for every 1% price hike.

The Design Matrix

What happened to `branddominicks`?

Our regression formulas look like $\beta_0 + \beta_1 x_1 + \beta_2 x_2 \dots$

But `brand` is not a number, so you can't do `brand  $\times$   $\beta$` .

The first step is to create a numeric *design matrix*.

It does this with a call to the `dmatrix` function:

	Intercept	brand[T.minute.maid]	brand[T.tropicana]	log_price
0	1.0	0.0	1.0	1.160021
1	1.0	1.0	0.0	1.026042
2	1.0	0.0	0.0	0.329304
3	1.0	0.0	1.0	1.078410
4	1.0	1.0	0.0	0.524729

The dummy variable is on the left, and on the right we have numeric x that we can multiply against β coefficients.

Intercepts

The OJ used `dmatrix` to build this 4 column design:

```
dmatrix(' ~ log_price + brand', data = oj)
```

Intercept	branBD	inute.maid	brandtropicana	log(price)
1.00000	0.000000		1.000000	1.160021
1.00000	1.000000		0.000000	1.026042
1.00000	0.000000		0.000000	0.329304
1.00000	0.000000		1.000000	1.078410
1.00000	1.000000		0.000000	0.524729

Each factor's **reference level** is absorbed by the intercept. Coefficients are 'change relative to reference' (dominicks here).

Why not include all three dummies?

Interaction

Beyond additive effects: variables change how others act on y .

An **interaction** term is the product of two covariates,

$$\mathbb{E}[y \mid \mathbf{x}] = \dots + \beta_j x_j + x_j x_k \beta_{jk}$$

so that the effect on $\mathbb{E}[y \mid \mathbf{x}]$ of a unit increase in x_j is $\beta_j + x_k \beta_{jk}$. **It depends upon x_k !**

Interactions play a massive role in statistical learning, and they are often central to social science and business questions.

- ▶ Does gender change the effect of education on wages?
- ▶ Do patients recover faster when taking drug A?
- ▶ How does advertisement affect price sensitivity?

Fitting interactions in Python: **use * in your formula.**

```
reg_interact = smf.ols(formula='log_sales ~ log_price * brand', data=oj).fit()
```

Coefficients:

(Intercept)	log(price)
10.95468	-3.37753
branBDminute.maid	brandtropicana
0.88825	0.96239
log(price):branBDminute.maid	log(price):brandtropicana
0.05679	0.66576

This is the model $E[\log(v)] = \alpha_b + \beta_b \log(\text{price})$: a separate intercept and slope for each brand 'b'.

Why not separately estimate three models?

Elasticities are dominicks: -3.4, minute maid: -3.3, tropicana: -2.7.

Where do these numbers come from? Do they make sense?

Advertisements

A key question: what changes when we feature a brand?

Here, this means in-store display promo or flier ad.

You could model the additive effect on log sales volume

$$\mathbb{E}[\log(v)] = \alpha_b + \mathbb{1}_{[\text{feat}]} \alpha_{\text{feat}} + \beta_b \log(p)$$

Or this *and* its effect on elasticity

$$\mathbb{E}[\log(v)] = \alpha_b + \beta_b \log(p) + \mathbb{1}_{[\text{feat}]} (\alpha_{\text{feat}} + \beta_{\text{feat}} \log(p))$$

Or its *brand-specific* effect on elasticity

$$\mathbb{E}[\log(v)] = \alpha_b + \beta_b \log(p) + \mathbb{1}_{[\text{feat}]} (\alpha_{b,\text{feat}} + \beta_{b,\text{feat}} \log(p))$$

See the Python code for runs of all three models.

Connect the regression formula and output to these equations.

Brand-specific Elasticities

	<i>Dominicks</i>	<i>Minute Maid</i>	<i>Tropicana</i>
<i>Not Featured</i>	-2.8	-2.0	-2.0
<i>Featured</i>	-3.2	-3.6	-3.5

Ads always decrease elasticity.

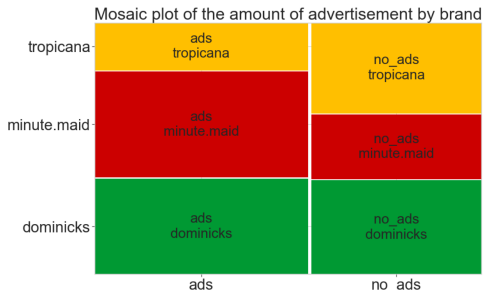
Minute Maid and Tropicana elasticities drop 1.5% with ads, moving them from less to more price sensitive than Dominicks.

Why does marketing increase price sensitivity?

And how does this influence pricing/marketing strategy?

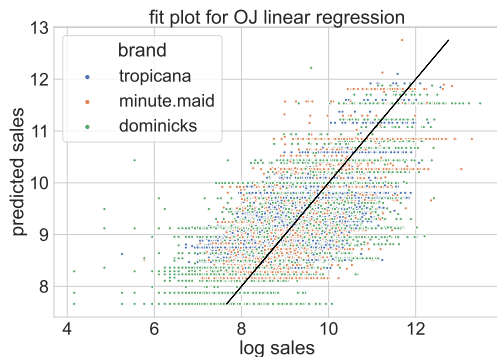
Confounding

Before including `feat`, Minute Maid behaved like Dominicks. With `feat`, Minute Maid looks more like Tropicana. Why?



Because Minute Maid was more heavily promoted, and promotions have a negative effect on elasticity, we were *confounding* the two effects in the brand average elasticity.

Fit Plot: $\hat{y}$ vs y



It's good practice to plot $\hat{y}$ vs y as a check for misspecification. (e.g., non-constant variance, nonlinearity in residuals, ...)